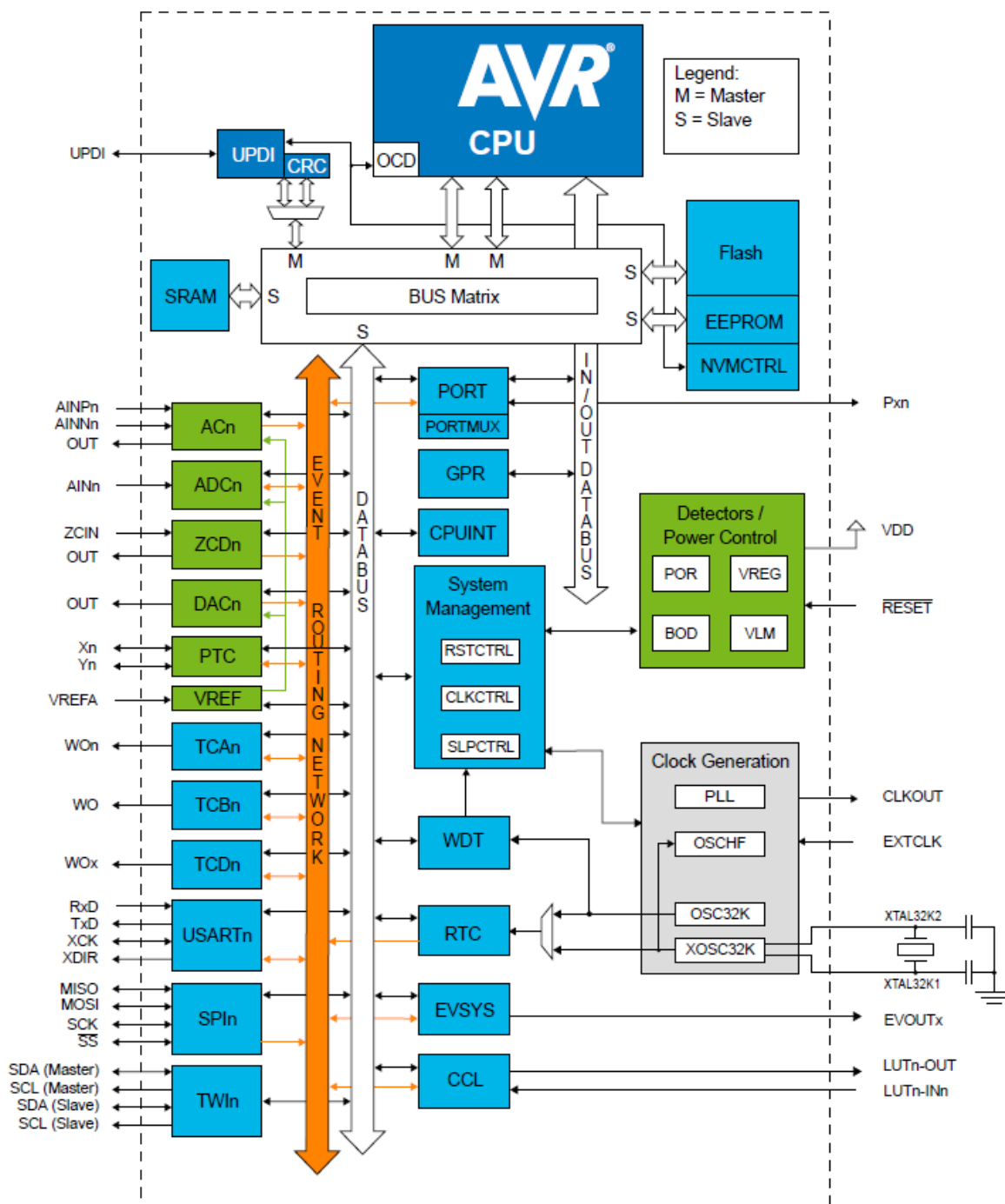


“Bare-Metal Embedded Systems with

AVR128DA48 Course”

Day 2 - AVR128DA48

Architecture Deep Dive



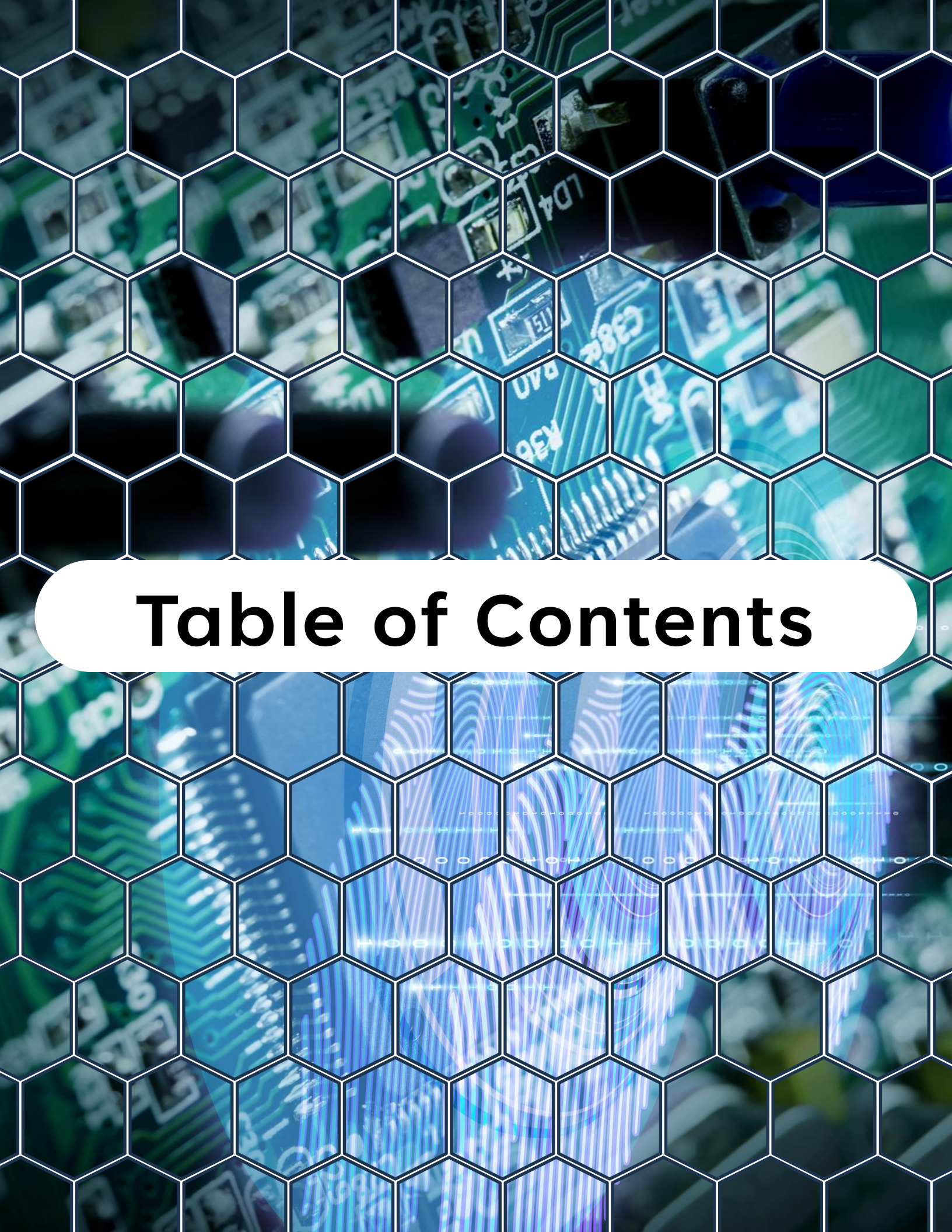
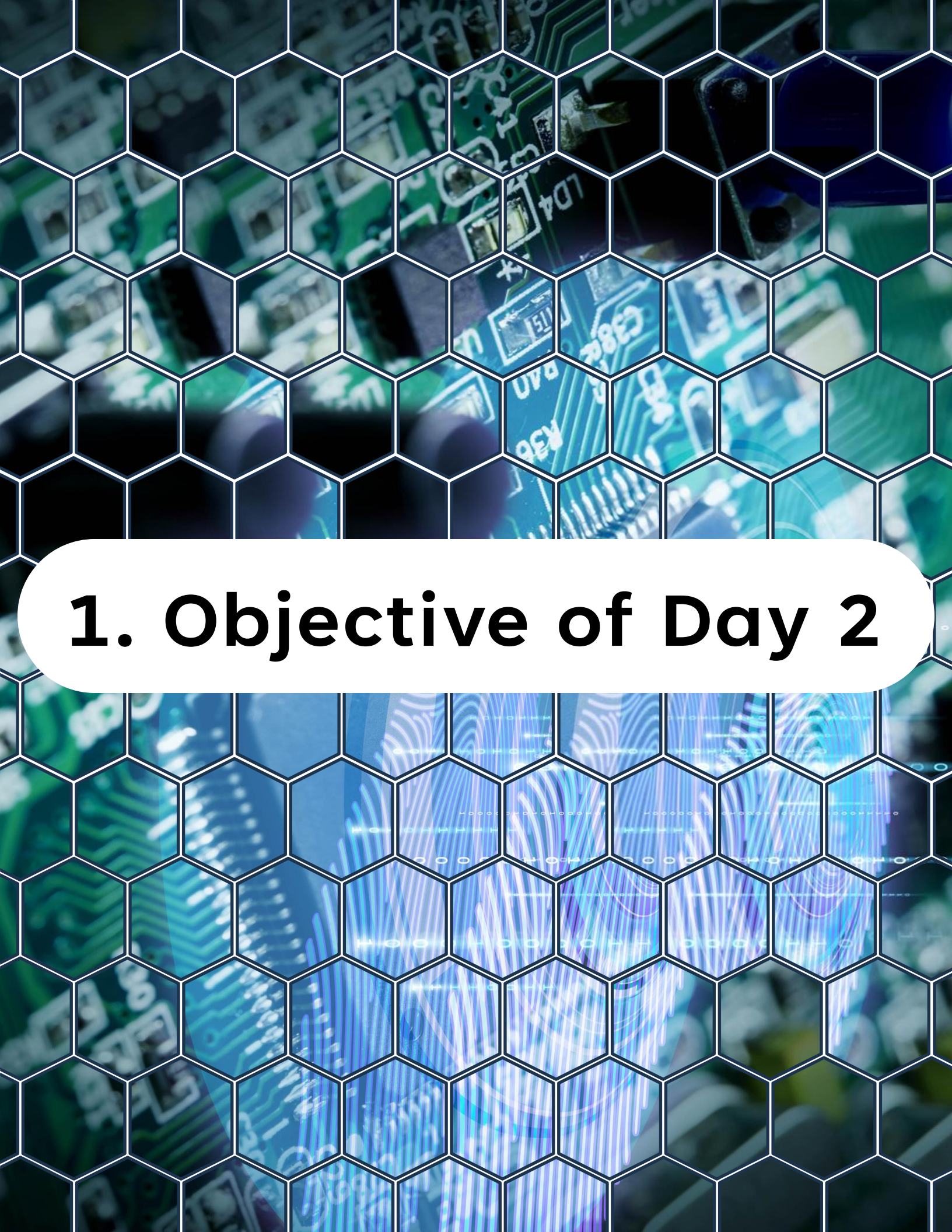


Table of Contents

Table of Contents

1. Objective of Day 2
2. High-Level MCU Architecture
3. CPU Core Overview
4. Harvard Architecture Explained
5. Memory Map Overview
6. General-Purpose Registers
7. I/O Memory and Peripheral Registers
8. SRAM Organization
9. EEPROM Architecture
10. Interrupt System Overview
11. Reset Sources and Startup Flow
12. Datasheet Navigation Strategy
13. Practical Exercise - Inspecting the Architecture
14. What You Learned Today
15. What Comes Next



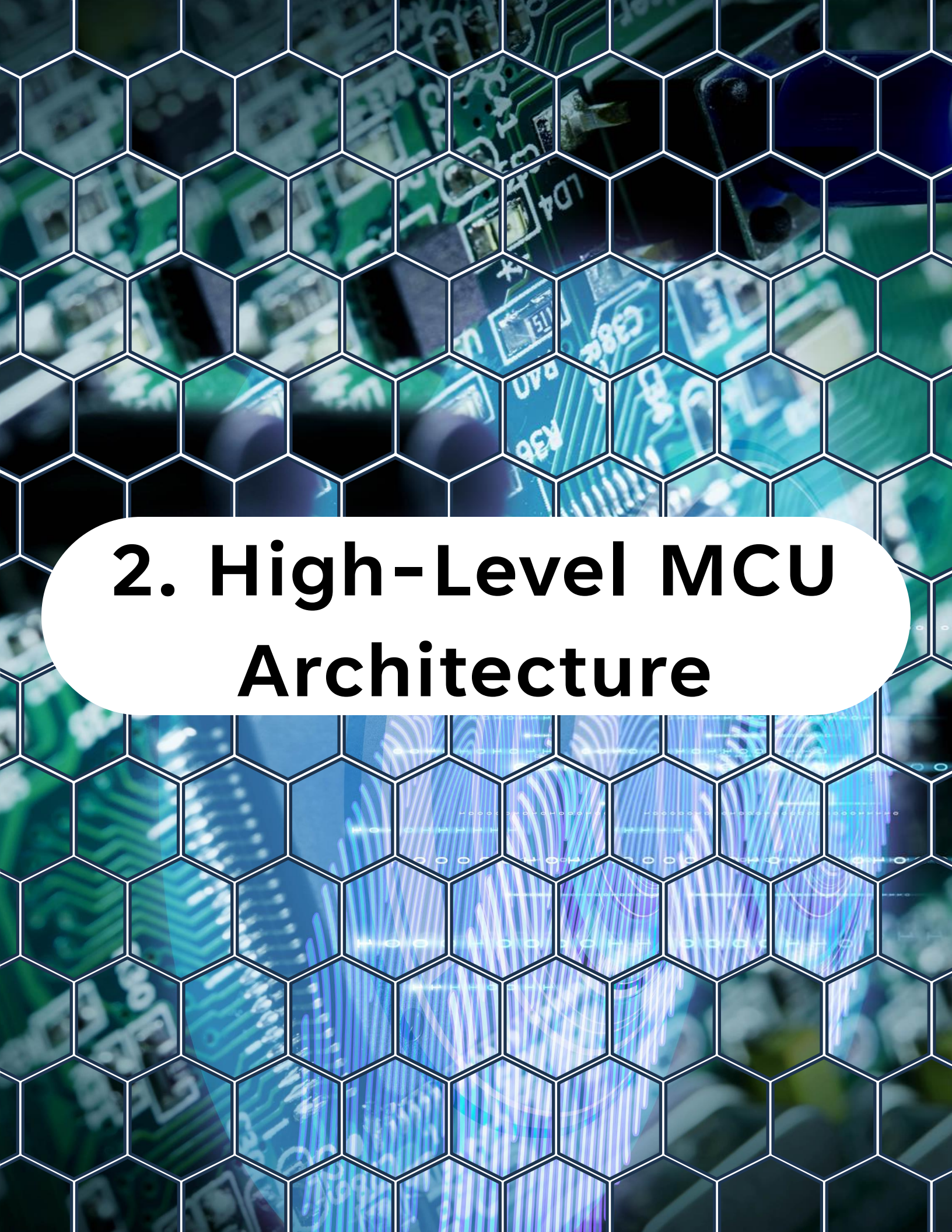
1. Objective of Day 2

1. Objective of Day 2

Day 2 is about understanding what actually exists inside the MCU and how the CPU sees it.

If you understand the architecture, everything else becomes easier: GPIO, timers, ADC, communication peripherals, debugging, and performance tuning. If you skip this step, you will always feel like you are guessing.

Today you will learn how the **AVR128DA48** is organized internally and how firmware interacts with hardware at the lowest level.



2. High-Level MCU Architecture

2. High-Level MCU Architecture

At a high level, the **AVR128DA48** consists of:

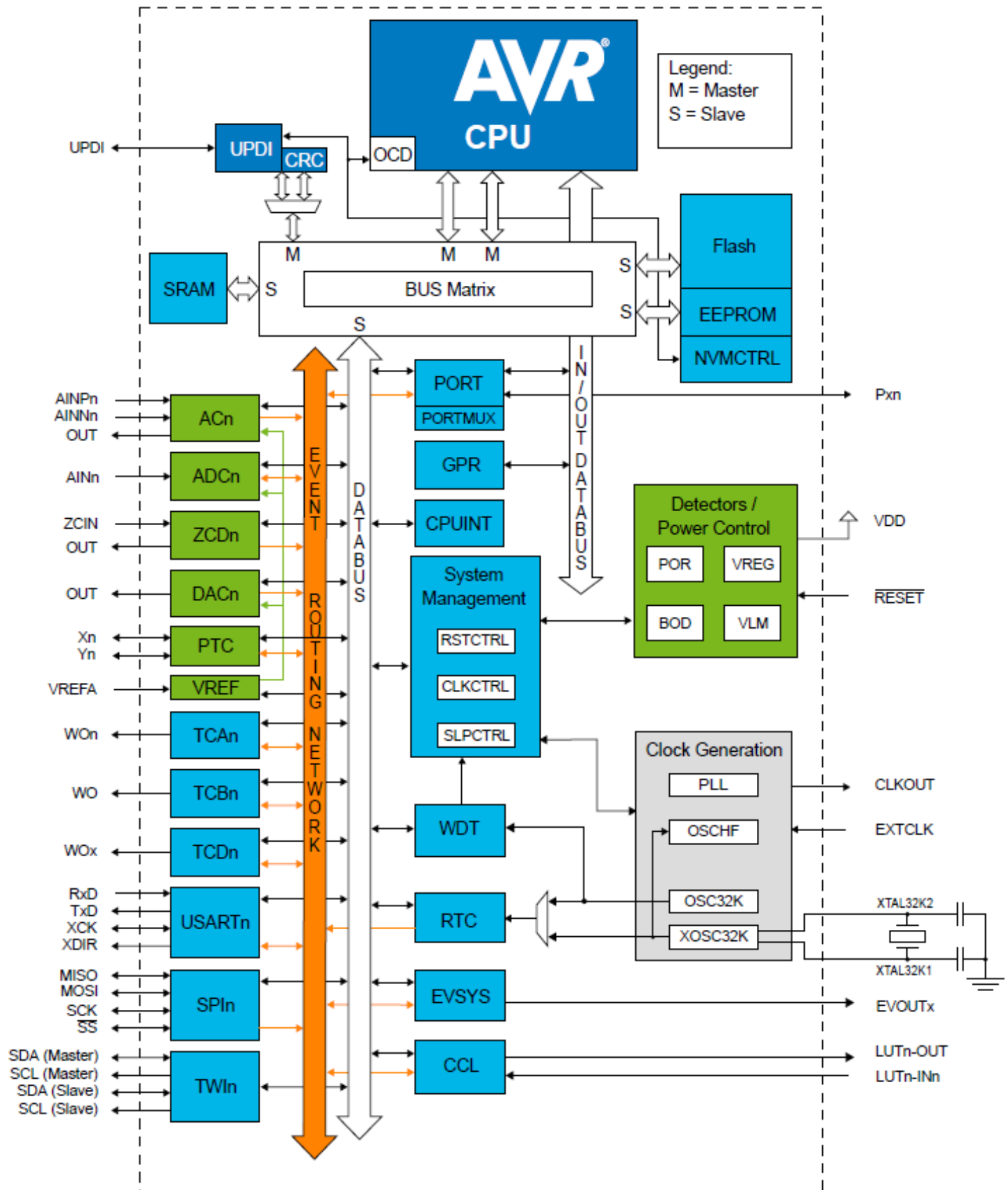
- CPU core
- Program memory (Flash)
- Data memory (SRAM, EEPROM)
- Memory-mapped peripherals (AC, ADC, DAC, TCA, USART, SPI, TWI)
- Interrupt controller
- Clock and reset system

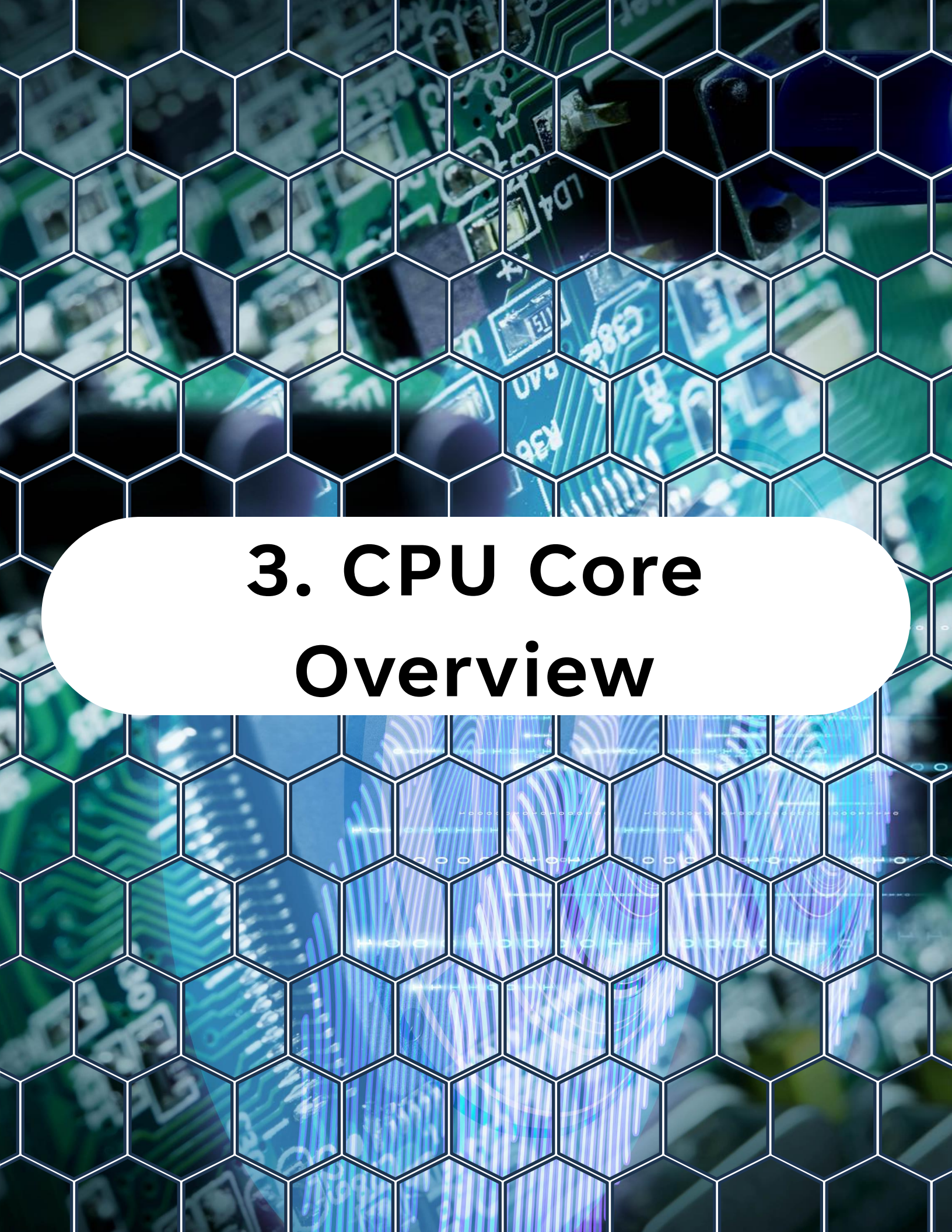
Everything is connected through a unified memory map. There is no magic. When you write to a register, you are writing to a memory address that controls real hardware.

This is the core idea behind bare-metal development.

2. High-Level MCU Architecture

Block Diagram





3. CPU Core Overview

3. CPU Core Overview

The **AVR128DA48** uses an 8-bit AVR CPU core with several modern improvements compared to older ATmega devices.

Key characteristics:

- 8-bit RISC architecture
- Single-cycle execution for most instructions
- 32 general-purpose working registers (R0 to R31)
- Separate instruction and data buses (Harvard architecture)

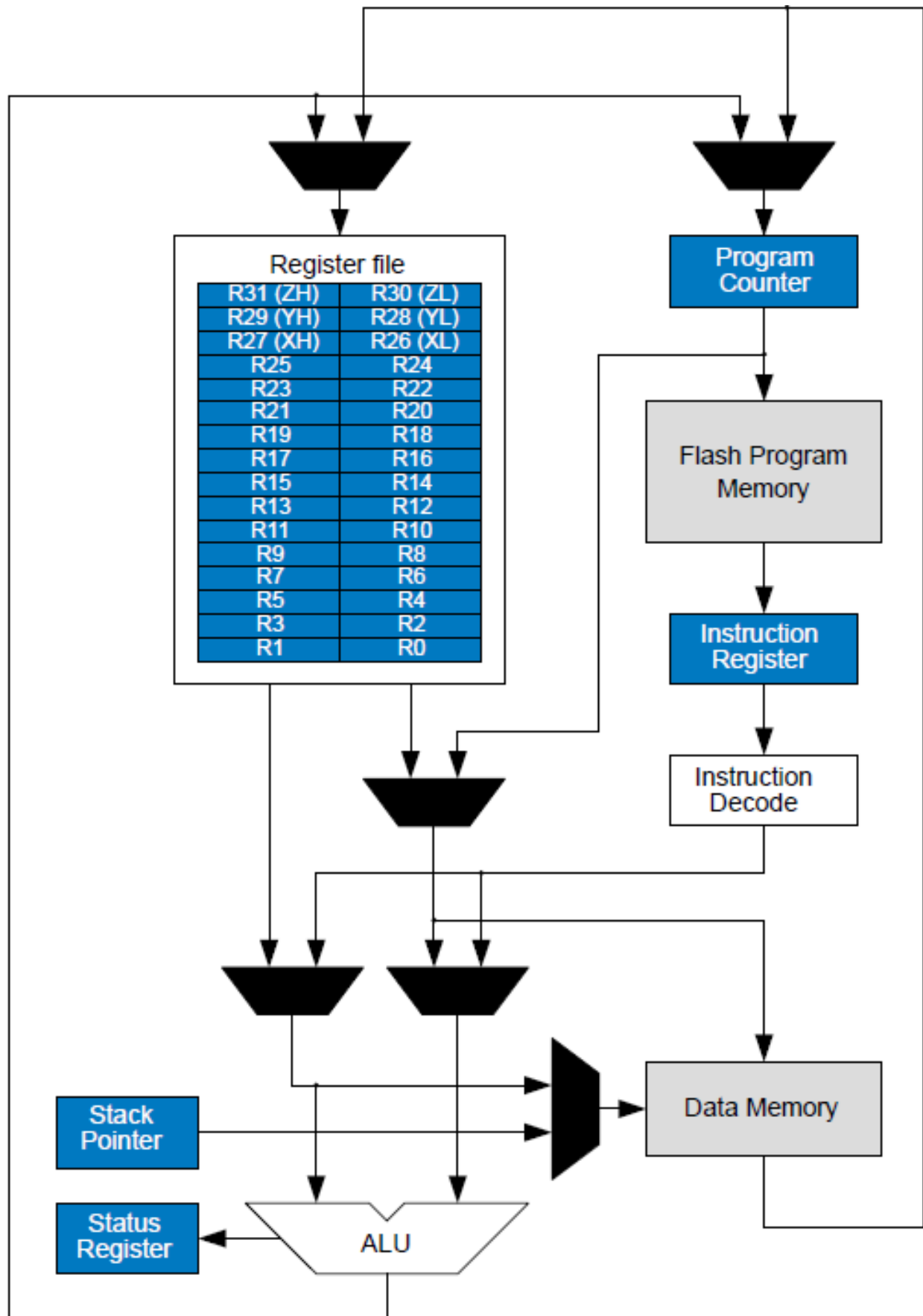
Why This Matters

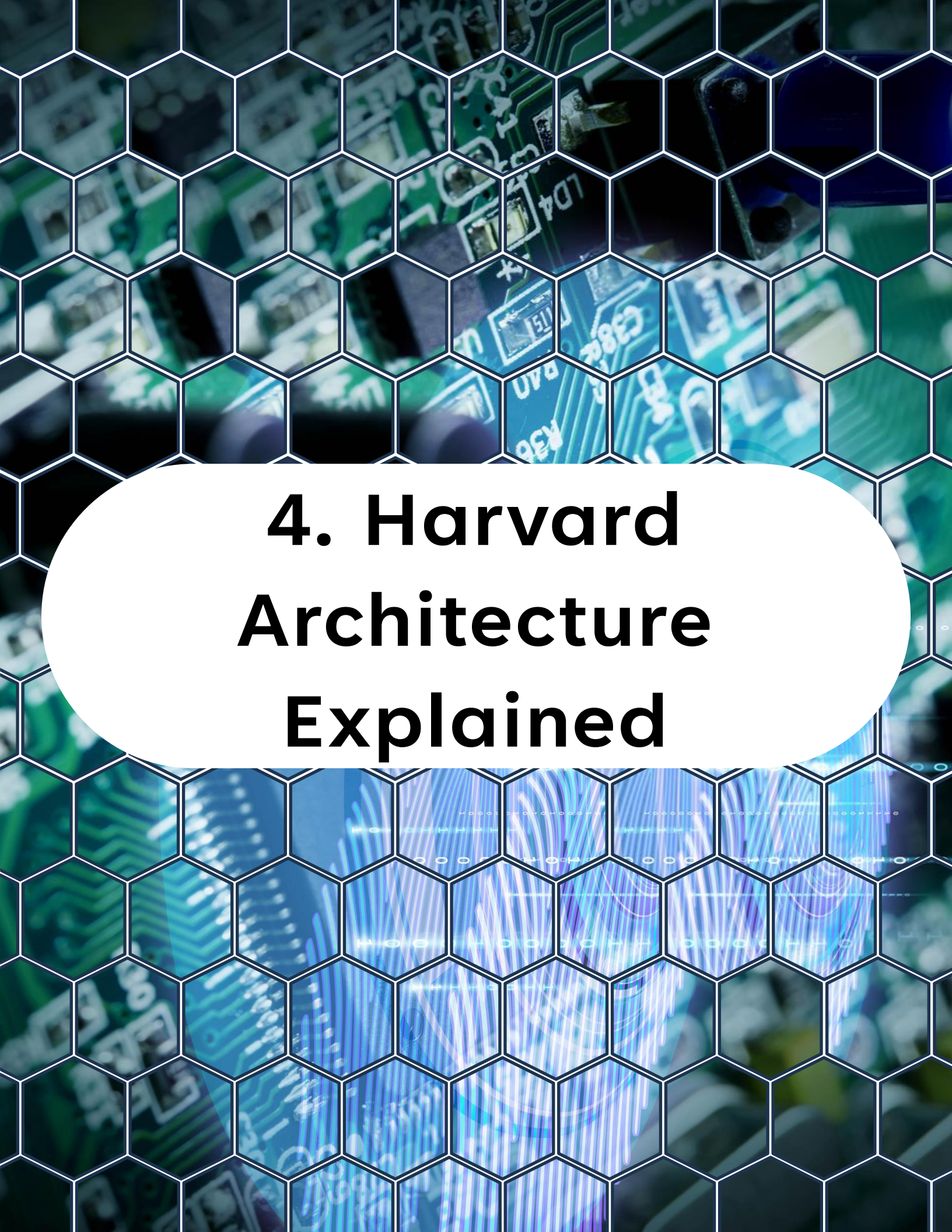
- Deterministic execution timing
- Very predictable interrupt latency
- Easy to reason about performance

When timing matters, simplicity wins.

3. CPU Core Overview

AVR® CPU Architecture





4. Harvard Architecture Explained

4. Harvard Architecture Explained

The AVR core uses a Harvard architecture, which means:

- Program memory (Flash) is separate from data memory
- Instructions are fetched from Flash
- Data is read and written from SRAM

This separation allows instruction fetch and data access to happen in parallel, improving performance even on a simple CPU.

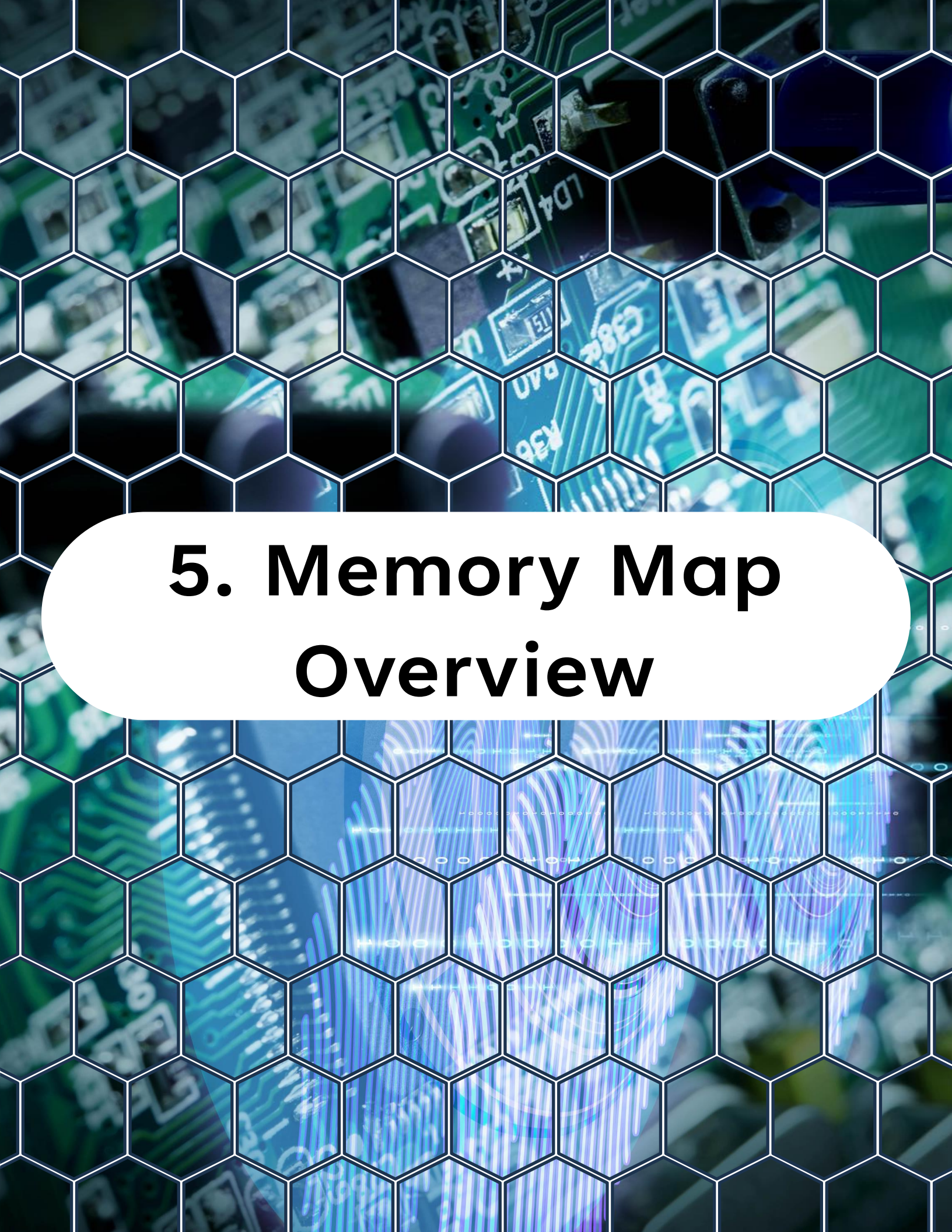
Practical Impact

- You cannot directly access Flash like RAM
- Special instructions are used to read program memory

4. Harvard Architecture Explained

- Constants placed in Flash behave differently than RAM variables

You will encounter this later when working with lookup tables and calibration data.



5. Memory Map Overview

5. Memory Map Overview

One of the most important concepts today is the memory map.

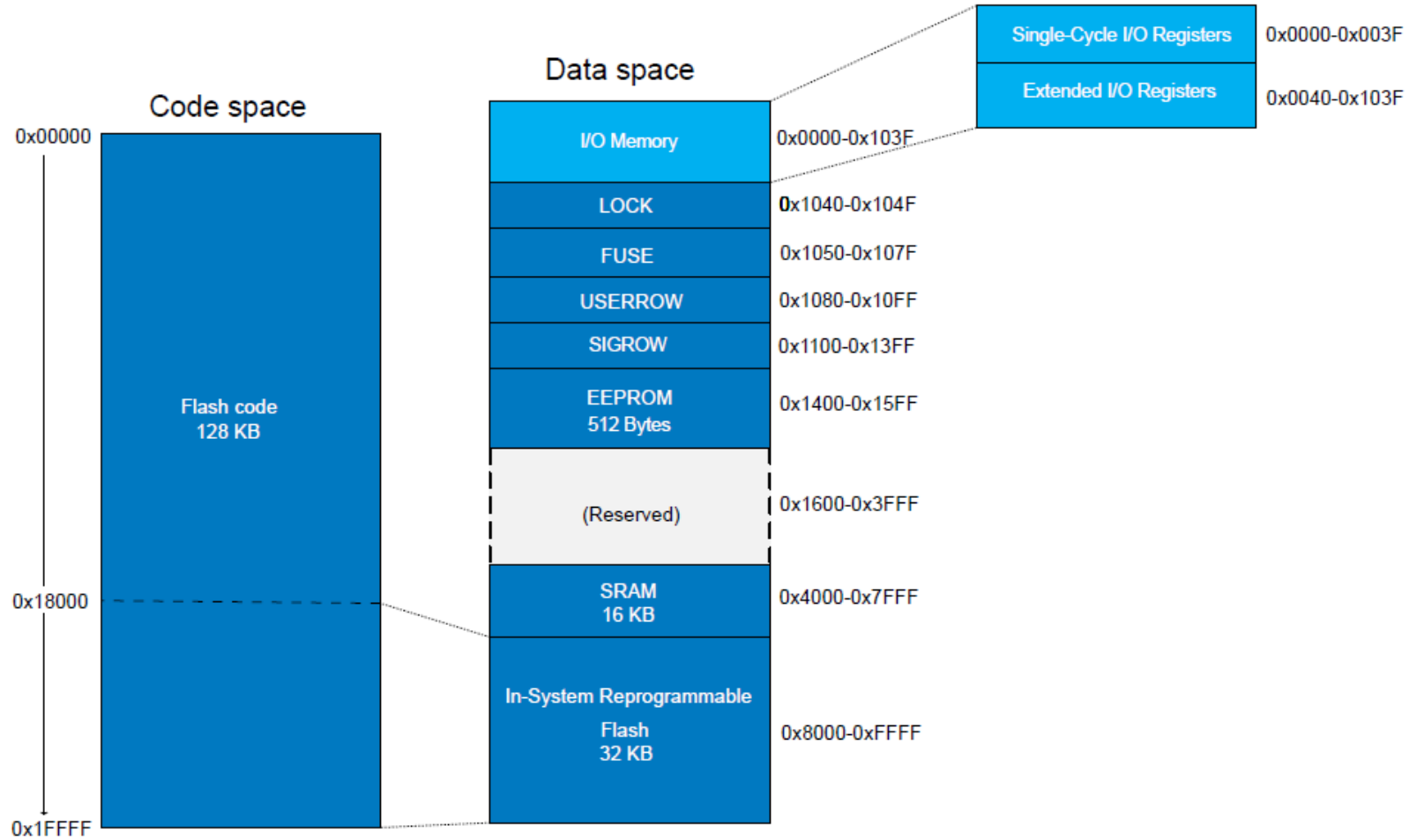
The **AVR128DA48** memory map is divided into:

- Register file
- I/O registers (peripherals)
- SRAM
- EEPROM (accessed via registers)
- Flash (program memory)

From the CPU point of view, peripherals are just memory locations.

5. Memory Map Overview

Memory Map





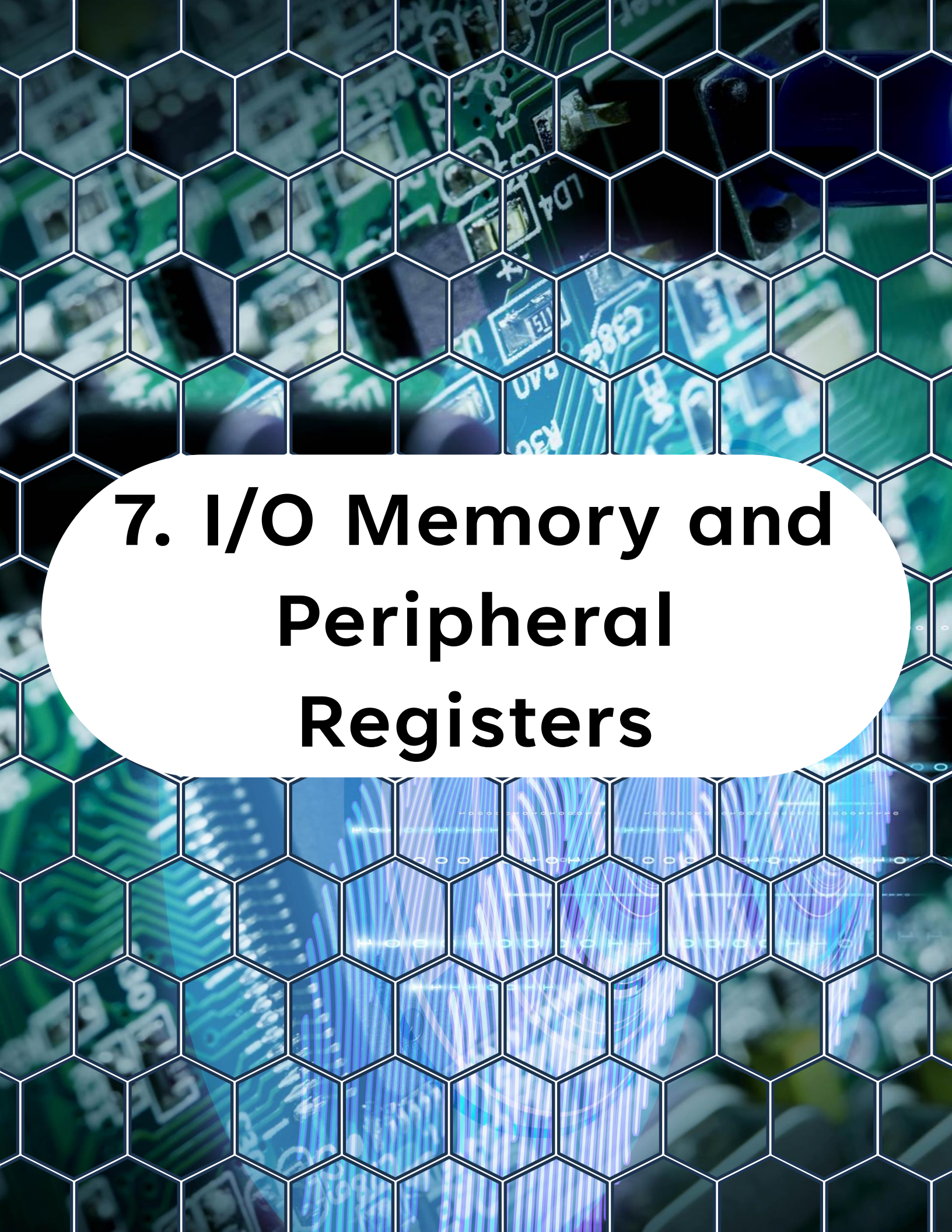
6. General-Purpose Registers

6. General-Purpose Registers

The first 32 bytes of data memory are the CPU registers:

- R0 to R31
- Used for arithmetic, logic, and pointer operations
- Compiler heavily optimizes around these

You normally do not manipulate these directly in C, but understanding they exist helps when reading assembly or debugging optimized code.

The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and resistors. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb effect. In the center, a white circular area contains the section title in bold black text.

7. I/O Memory and Peripheral Registers

7. I/O Memory and Peripheral Registers

After the register file comes I/O memory.

This is where all peripherals live:

- PORT registers
- TCA, TCB timer registers
- USART, SPI, I2C registers
- ADC, DAC, AC registers

Each peripheral has:

- Control registers
- Status registers
- Data registers

These registers are defined in `<avr/io.h>` as C structs and macros that map directly to hardware addresses.

7. I/O Memory and Peripheral Registers

Key Concept

When you write:

```
PORTA.DIR = 0xFF;
```

You are writing to a physical memory address connected to the PORTA hardware.

There is no software layer in between.



8. SRAM Organization

8. SRAM Organization

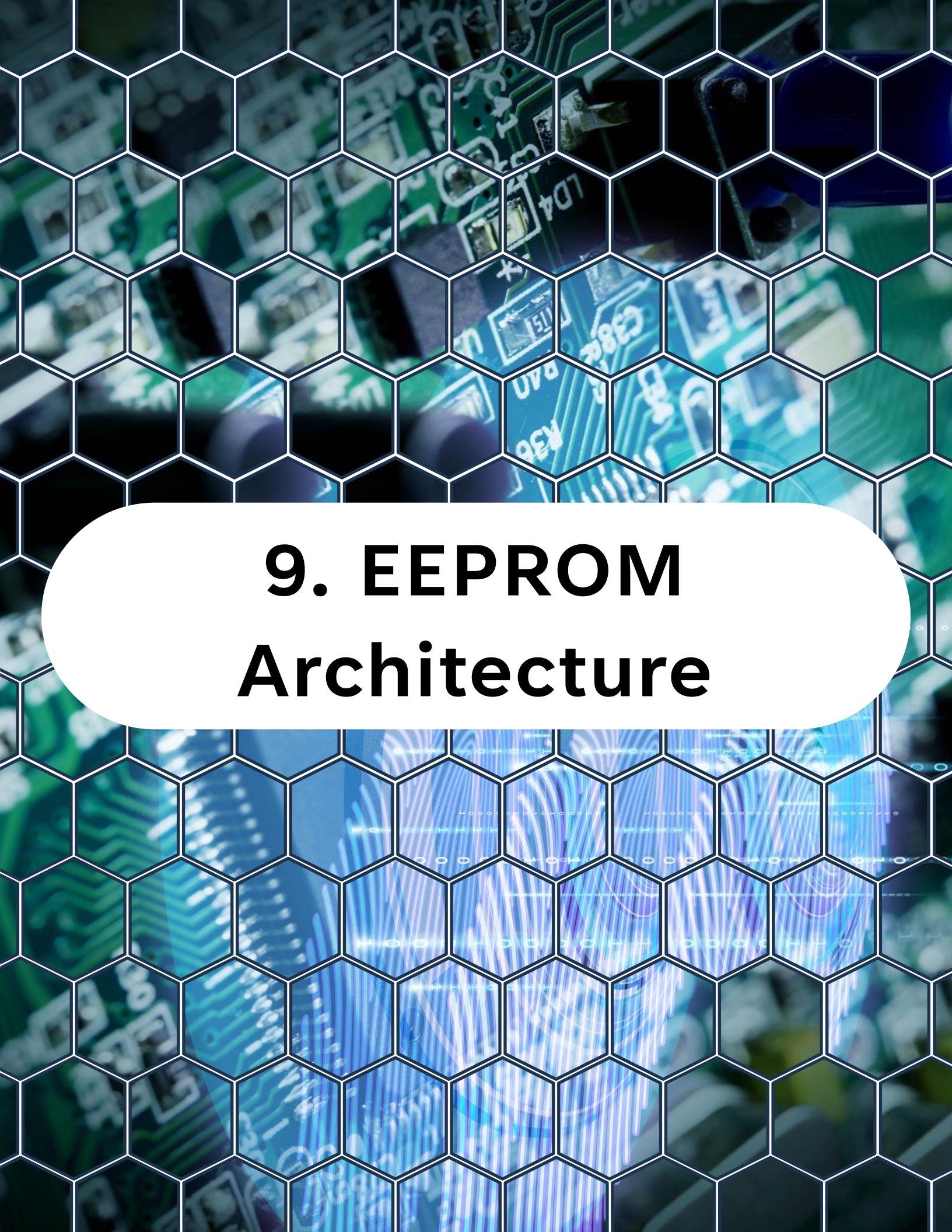
SRAM is where:

- Global variables live
- Stack is allocated
- Heap (if used) resides

Key points:

- Stack grows downward
- Global and static variables are allocated at startup
- Stack and heap share the same memory

On microcontrollers, stack overflow is silent and dangerous. Later in the course, you will learn how to design systems that avoid this problem entirely.



9. EEPROM Architecture

9. EEPROM Architecture

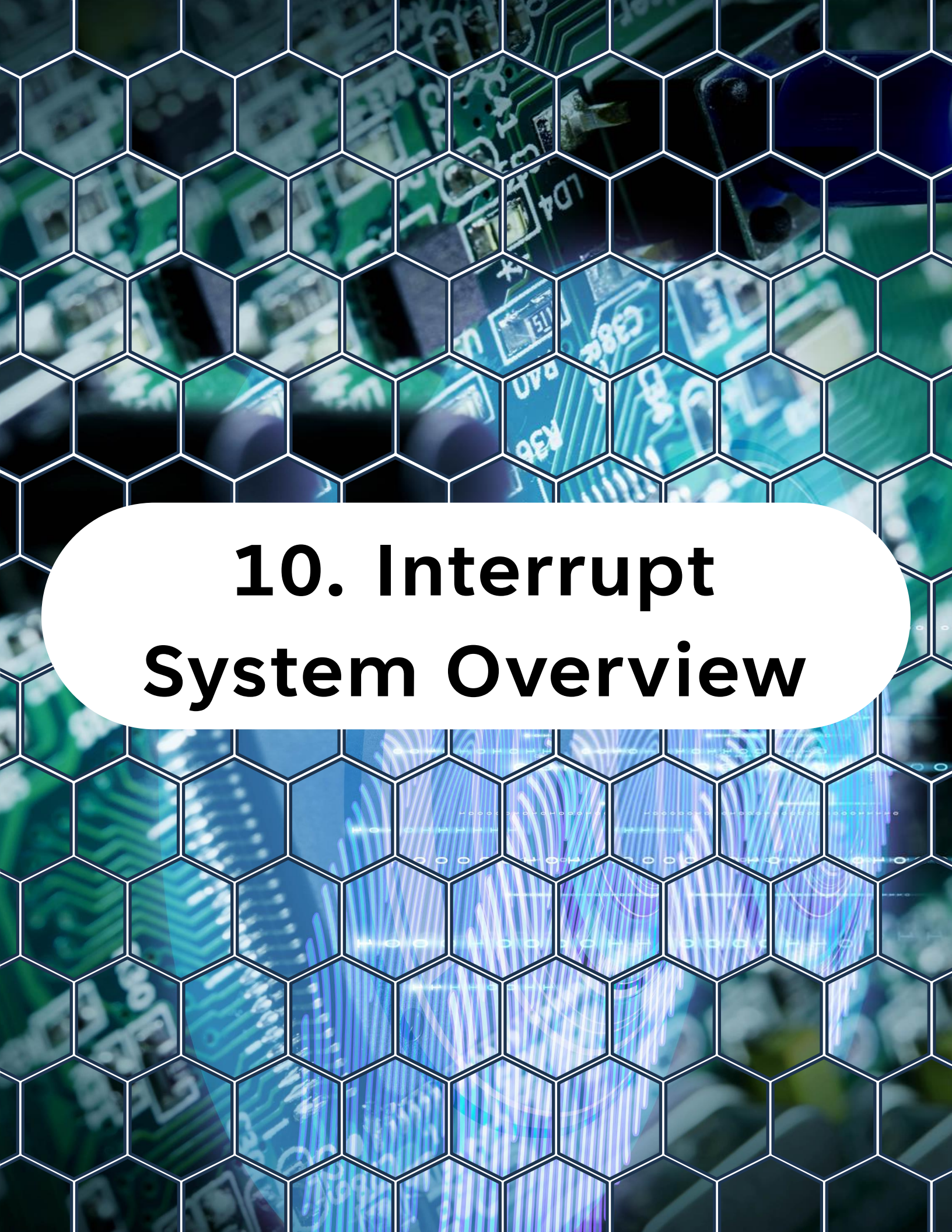
EEPROM is non-volatile memory used for:

- Configuration storage
- Calibration data
- Persistent settings

It is not memory-mapped like SRAM. Instead:

- Access is done through EEPROM control registers
- Writes are slower than RAM
- Limited write endurance

We will treat EEPROM with care and discipline when we reach it.



10. Interrupt System Overview

10. Interrupt System Overview

The **AVR128DA48** interrupt system is:

- Vector-based
- Fixed priority (by vector order)
- Fast and deterministic

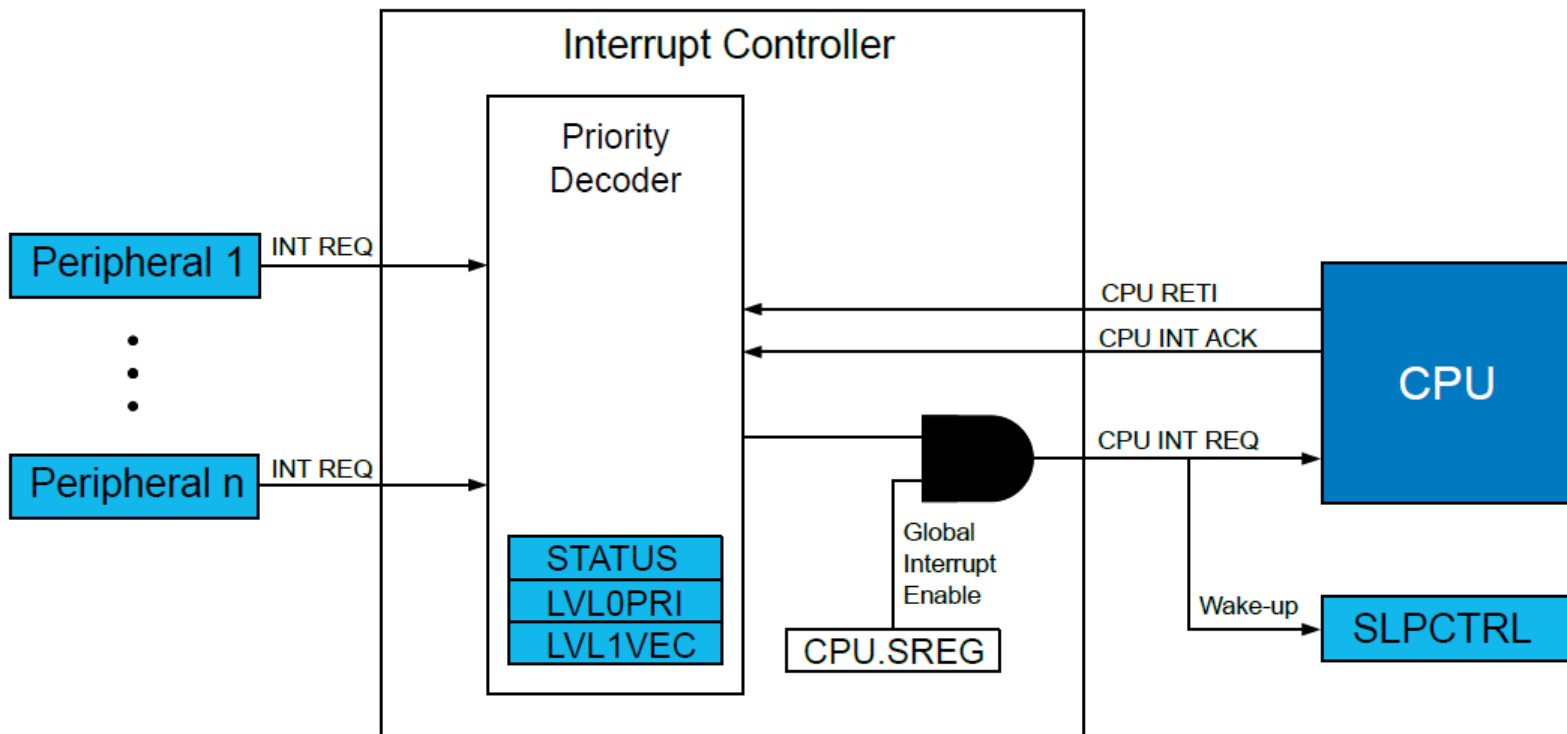
Each peripheral can generate interrupts that:

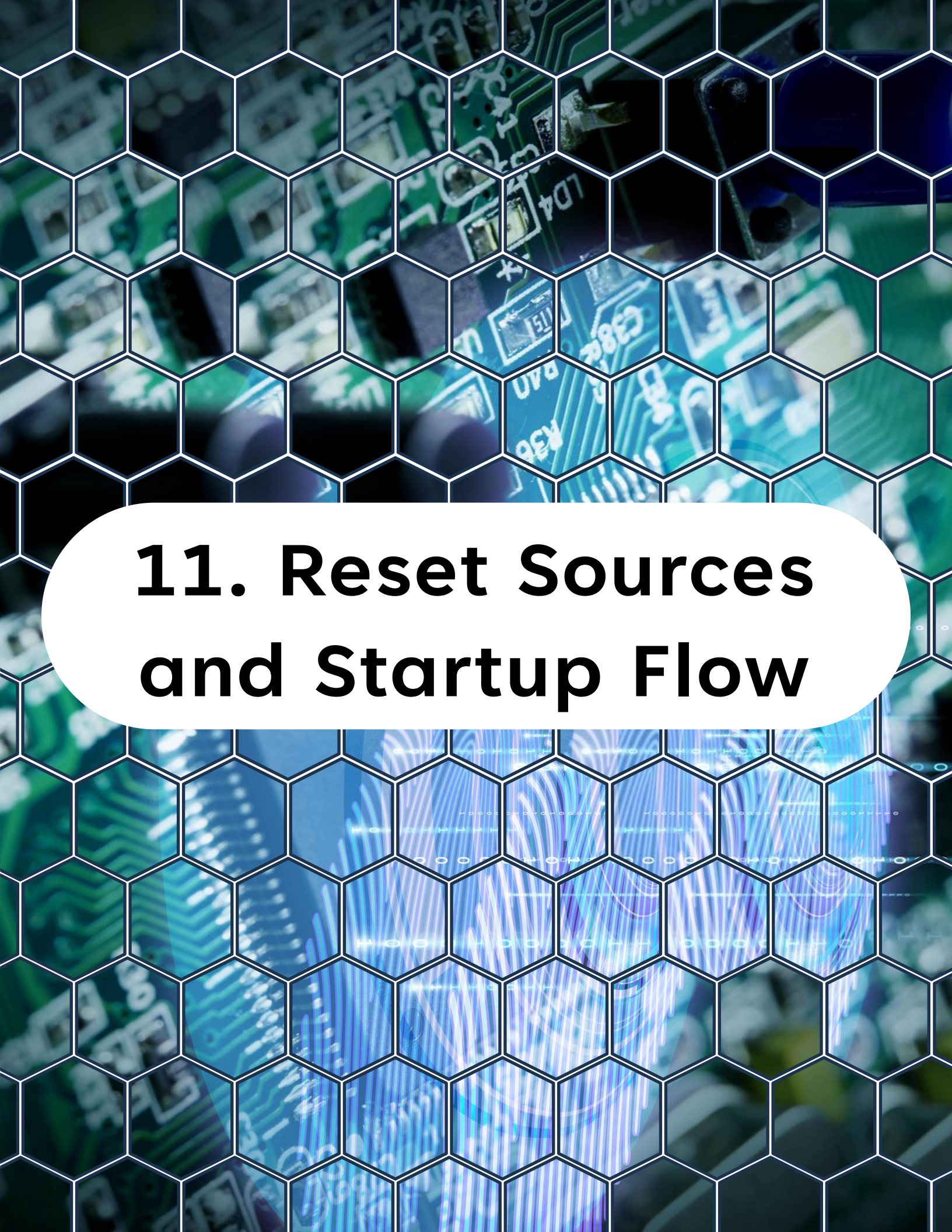
- Jump execution to a specific ISR
- Save CPU context automatically
- Resume execution after ISR completion

Understanding the interrupt flow early helps avoid many common mistakes later.

10. Interrupt System Overview

CPUINT Block Diagram





11. Reset Sources and Startup Flow

11. Reset Sources and Startup Flow

When the MCU resets, execution always starts at the reset vector.

Common reset sources:

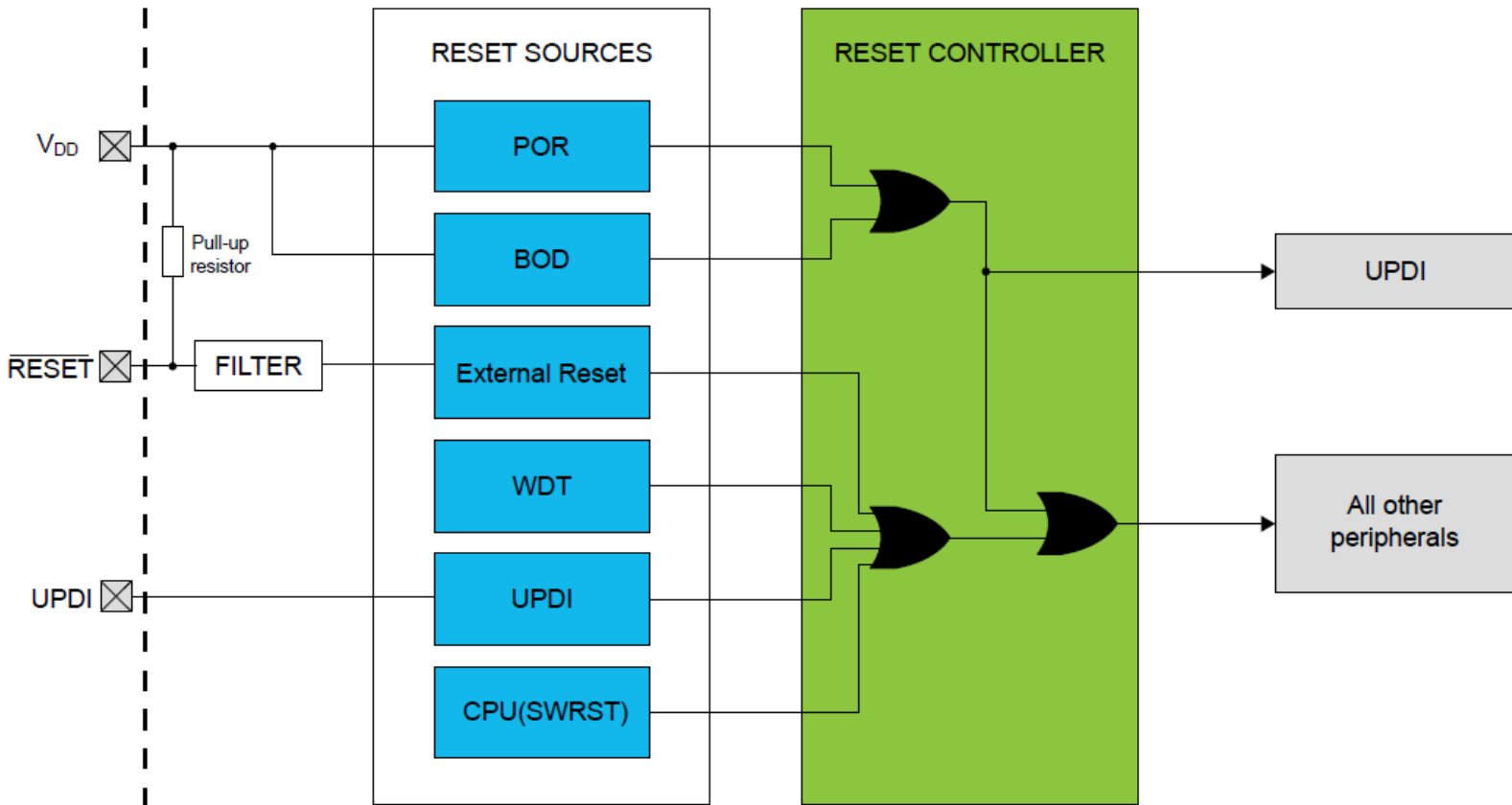
- Power-on reset
- External reset
- Brown-out reset
- Watchdog reset
- Software reset

The reset cause is stored in a status register.

Reading it early in firmware can be extremely useful for debugging and reliability.

11. Reset Sources and Startup Flow

Reset System Overview





12. Datasheet Navigation Strategy

12. Datasheet Navigation Strategy

You should now open the datasheet and locate:

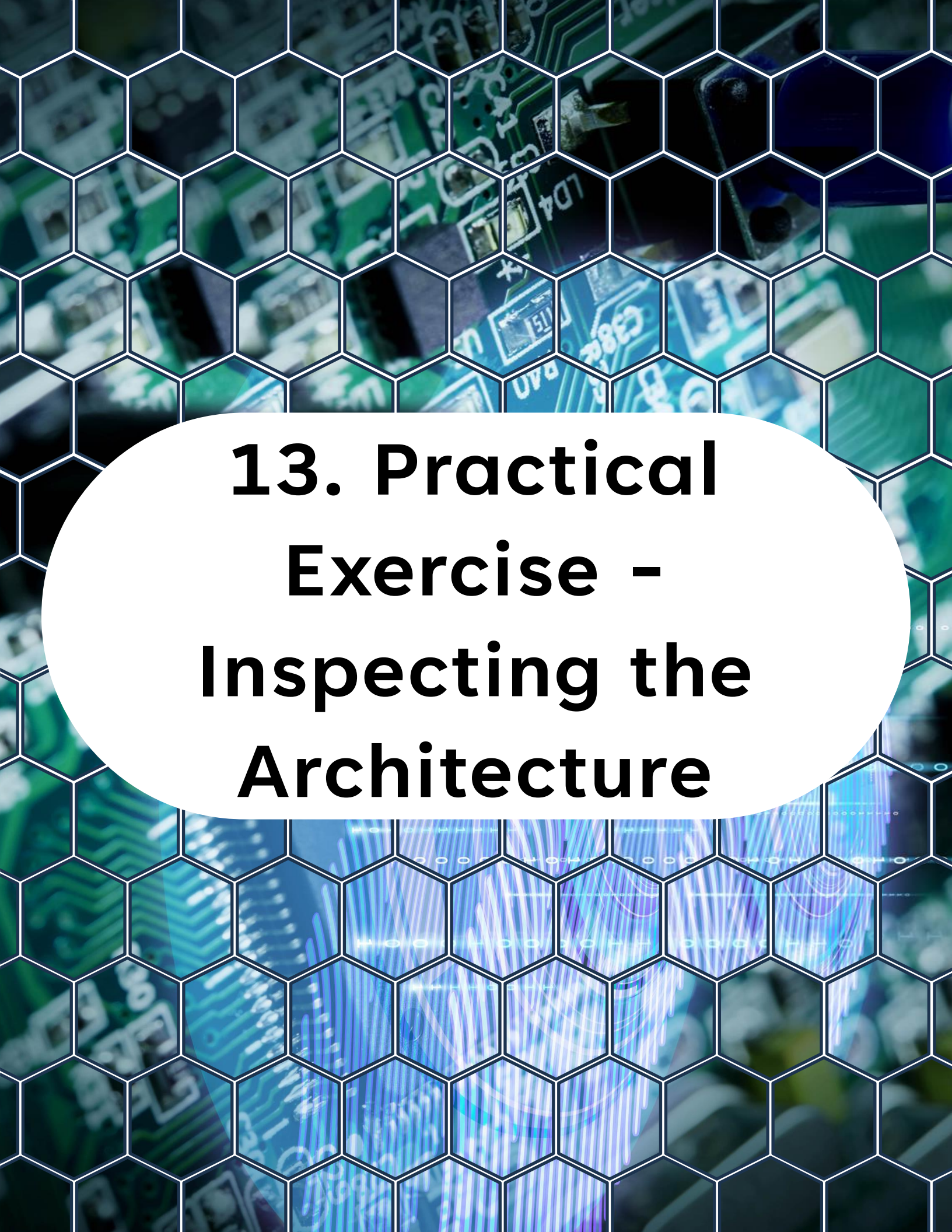
Block diagram

- Memory map section
- CPU overview
- Peripheral overview table

Do not try to read everything line by line. Learn to jump directly to what matters:

- Register descriptions
- Bit definitions
- Timing diagrams

This skill separates productive engineers from frustrated ones.



13. Practical Exercise - Inspecting the Architecture

13. Practical Exercise - Inspecting the Architecture

Exercise Goals

- Build confidence with the debugger
- Visualize memory and registers

Steps

1. Build and flash your Day 1 project
2. Start a debug session
3. Pause execution
4. Open:
 - Register view
 - Memory view
 - I/O register view



14. What You Learned Today

14. What You Learned Today

By the end of Day 2, you should understand:

- How the AVR CPU core works
- How memory is structured
- Why peripherals are memory-mapped
- How C code directly controls hardware
- How to navigate the datasheet efficiently

This knowledge will pay dividends every single day of this course.



15. What Comes Next

15. What Comes Next

Day 3 - Clock System and Clock Domains

Next, you will learn:

- How the CPU clock is generated
- How prescalers affect timing
- Why clocks impact power and accuracy
- How to prove clock settings using real measurements

From Day 3 onward, your firmware will start interacting visibly with the outside world.

13. Practical Exercise - Inspecting the Architecture

Observe how:

- Registers change
- SRAM is initialized
- Peripherals appear as memory locations

No code changes today. Observation is the lesson.

"Thanks for watching!

If you enjoyed this video, make sure to hit the like button and subscribe to stay updated with my latest content.

Don't forget to check out my other videos for more tips and tutorials on Embedded C, Python, hardware designs, etc. Keep exploring, keep learning, and I'll see you in the next video!"

<https://www.linkedin.com/in/yamil-garcia>

<https://www.youtube.com/@LearningByTutorials>

<https://github.com/god233012yamil>